

OpenBedside: an open-source gateway for bedside devices

What it is, and how to try it. Version 0.2.0, 26 September 2026. Daniel C. Pettus.

In one sentence: a free, open-source gateway that takes data from bedside IV infusion pumps and delivers it to the hospital EHR in standard HL7, so anyone can connect a device by writing one adapter and get store-and-forward, EHR connectivity and an interface activity page for free.

Why I built it

My reason is a selfish one, and I would rather say so up front.

I am building **AI MedAgent**, a medication reasoning engine (aimedagent.net; the public demonstration runs on a simulated patient). A reasoning engine is only as good as what it can see, and it needs two feeds: the EHR and the bedside devices.

The EHR half has a door now. Federal certification rules require certified EHR technology to offer a standardized FHIR API (ONC, 45 CFR 170.315(g)(10)). Access is not free and not always quick, but the door exists and it looks much the same from one EHR to the next.

The device half has no door. Every infusion pump that reports to an EHR does it through its manufacturer's own gateway, in its own protocol, and each one is closed. IV infusions are one of the most important feeds a medication reasoning engine can have, because that is where the drug actually enters the patient, and they are the hardest data to get.

So I am doing the selfish thing in the most useful way I know: opening the device half for everyone, me included. OpenBedside is a **free, open-source device gateway. No fees, no royalties, no permission needed.** It starts with IV infusion pumps because that is the feed I need most, and it is built to take ventilators and other devices next.

OpenBedside contains no AI MedAgent code and no clinical reasoning. It is plumbing. It sends standard HL7 to any hospital system that wants it. AI MedAgent is one possible consumer of that data, and not a required one.

What it is

- **The internal logic.** A common data model for a pump: device, modules, channels, primary and secondary infusions. It tracks state and works out events from what the device reports: start, stop, complete, rate change, KVO, secondary to primary.
- **Store-and-forward.** Every message is saved to disk before it is sent, delivered in order, and retried until the hospital acknowledges it. A restart or an outage loses nothing.
- **EHR connectivity.** Standard IHE Devices HL7 v2 messages over MLLP, the form hospital interface engines already expect.
- **Device registration.** Nothing is sent until a device is registered, by vendor and serial. The registration says what the nurse scans (serial, asset tag or EUI-64), where the device lives, and how its patient may be found. Pumps and ventilators can both be registered.
- **Patient association.** From the device itself (a wristband scanned at the pump), the hospital ADT feed for the device's bed, or a person. The sources must agree, or no patient is sent. A patient change from ADT while a pump is running waits for a person to confirm, because that is how wrong-patient charting starts.
- **A web page.** Activity: a live view of every device, connection, event and message. Devices: registration. Patients: association and the ADT census. Sign-in protects every change.

- **An adapter kit.** A base class, a complete worked example, a checker that finds mapping mistakes before a hospital does, and a command that writes a starter folder for your device.

Open source, for anyone

Nothing in OpenBedside is proprietary. It is released under the **Apache License 2.0**. Anyone may download it, use it, test it, change it, build it into their own product, and extend it to other devices. Improvements are welcome back, and your own device adapter can stay private if you prefer; the license allows both.

It was designed from public sources: the IHE Devices Technical Framework, ISO/IEEE 11073 nomenclature as published, FDA guidance, and experience. No manufacturer's protocol or code is in it. The worked example uses a fictional pump with an invented protocol.

What you write for a new device

Whether you are a manufacturer, a hospital integration team or a researcher, you write one thing: an **adapter**. Every pump reports the same facts in different words, over a different connection. The adapter is the translation. In the worked example it is about 110 lines of Python:

Part	What it does
Vocabulary tables	Your state names and unit strings mapped to OpenBedside's, in one place
Transport	connect and read_message: TCP, serial, your own device server, whatever your device uses
Translation	normalize: one raw message in, one standard snapshot out

Everything after that, from state and events to delivery and the activity page, is identical for every device.

The web page

The worked example running: the fictional ACME pump registered by its asset tag to ICU 204 bed 1, its patient found from a synthetic ADT admit, two modules infusing, a piggyback nearly finished, and every message delivered to the stand-in hospital system and acknowledged.

OpenBedside Activity Devices Patients this computer: changes allowed v0.2.0

Not a medical device. Not for clinical use. Test data only: use synthetic patient identifiers, this release has no encryption. Auto-programming works against the simulator only.

No admin password is set, so changes are allowed only from a browser on this computer. Set one with `openbedside set-password`.

STATUS
EHR link: **connected** Queue: sent 2

DEVICE CONNECTIONS (ADAPTERS)

Adapter	Connection	Messages	Skipped	Last error
acme-json (example)	connected	9	0	

DEVICES

Device	Patient	Module	Status	Actual rate	Sources
BIO-004512 ACME (fictional) ACME-2 · ICU 204 1	TEST0042 adt-location	ACME-7731-L	infusing	12 mL/h	primary heparin 12 mL/h (1200 [iU]/h) VTBI 250 mL, remaining 246.8 mL, delivered 3.2 mL
BIO-004512 ACME (fictional) ACME-2 · ICU 204 1	TEST0042 adt-location	ACME-7731-R	infusing	100 mL/h	primary sodium chloride 0.9% 75 mL/h VTBI 500 mL, remaining 500 mL, delivered 0 mL secondary cefazolin 100 mL/h VTBI 50 mL, remaining 23.333 mL, delivered 26.667 mL

EVENTS

When (UTC)	Event	Where	Detail
17:46:26	patient-associated derived	ACME-7731	{"patient": "TEST0042", "source": "adt-location"}
17:46:23	delivery-start derived	ACME-7731-R	{"note": "first observation already infusing"}
17:46:23	delivery-start derived	ACME-7731-L	{"note": "first observation already infusing"}

OUTBOUND TO EHR

#	Control id	Status	Tries	Last result
2	PD2026092617462600002	sent	1	AA
1	PD2026092617462300001	sent	1	AA

ORDERS RECEIVED (PCD-03)
None.

Registration, on the Devices page:

OpenBedside Activity Devices Patients this computer: changes allowed v0.2.0

Not a medical device. Not for clinical use. Test data only: use synthetic patient identifiers, this release has no encryption. Auto-programming works against the simulator only.

No admin password is set, so changes are allowed only from a browser on this computer. Set one with `openbedside set-password`.

SEEN BUT NOT SENT
Devices an adapter can hear that are not registered, are retired, or are registered as a different type. Nothing from them reaches the EHR.
None.

REGISTERED DEVICES

Type	Vendor	Device id	EHR id	Location	Patient from	Status	
Infusion pump	ACME (fictional)	ACME-7731 ACME-2	BIO-004512 asset	ICU 204 1	device, ADT, manual	active	<button>Edit</button> <button>Retire</button>

REGISTER A DEVICE

Device type
Infusion pump

Vendor (as the adapter reports it)
ACME (fictional)

Device id (serial, as the device reports it)
ACME-7731

Model
ACME-2

Hospital asset tag
BIO-004512

EUI-64 (16 hex characters, if it has one)

Send to the EHR as (what the nurse scans)
Asset tag

Unit (point of care)
ICU

Room
204

Bed
1

Facility

Connection binding (only for devices that never send an id)
adapter:port

Find this device's patient from
☒ the device (wristband scanned at the device)
☒ ADT: the patient in this device's bed
☒ a person, on the Patients page

Notes

Save Clear
Saved.

Try it in fifteen minutes

1. Install Python 3.11 or newer (free, from python.org).
2. Download OpenBedside from github.com/danielpettus/openbedside (the kit is under Releases), and follow `START-HERE-install-on-your-Mac.md` inside it. Linux is the same, and Windows is covered in the user guide.
3. Run **openbedside demo** and open `http://127.0.0.1:8080`. A simulated pump talks to a stand-in hospital system.
4. Run the worked example: **openbedside fake-pump**, **openbedside ehr-sim**, then **openbedside run -c config/acme-example.toml**. Register the ACME pump on the Devices page and watch its data start to flow.

Connecting your device

The vendor integration guide (`docs/VENDOR-INTEGRATION-GUIDE.md`) walks through it step by step:

1. Fill in the **mapping worksheet**: transport, identity, every state, every value and unit, piggybacks, and what your device does not send.
2. Run **openbedside new-adapter yourcompany** to get a starter folder with the adapter, a config, tests and the worksheet.
3. Write the four marked sections of the adapter.
4. Run **openbedside check**. It runs your adapter against your device, sends nothing, reports every problem, and shows the exact HL7 message a hospital would receive. Get to PASS.
5. Run against the stand-in hospital and watch the activity page.
6. Work through the 14 hard cases in the guide: pause, piggyback, KVO, power loss, network loss, hospital system down, gateway restart, clock drift.
7. Take the results, the HL7 interface document and your completed worksheet to a hospital's interface team, and connect to their **test** system.

What it is not

Not a medical device and not for clinical use. Status reporting, store-and-forward and message conversion are data functions. Sending programs to a pump and delivering alarms to clinicians are regulated device functions: in this release, auto-programming works only against the simulator, and alarm delivery is out of scope. **Synthetic patient data only.** Once real identifiers flow through it, the gateway holds protected health information and needs encryption on its HL7 links, web page and database, and an access log. None of those is in this release yet. Anyone who builds OpenBedside into a product takes on that product's regulatory obligations.

Beyond infusion pumps

The outbound message is not pump-specific: IHE's PCD-01 carries data from monitors, ventilators and infusion pumps alike, so a ventilator uses the same message and the same schema pattern. Registration already accepts ventilators. What changes is the data model, the device codes (IHE Devices Technical Framework Volume 3, section 7.2, Device: Ventilator), the containment tree and a simulator. Ventilator support is planned, and it is exactly the kind of contribution the project exists for.

What is in the kit

File or folder	What it is
README.md	Why it exists, what it is, commands, where to go next
START-HERE-install-on-your-Mac.md	Install and first run, step by step
docs/VENDOR-INTEGRATION-GUIDE.md	Connecting your device. The most important document.
docs/MAPPING-WORKSHEET.md	The worksheet to fill in before writing code
docs/USER-GUIDE.md	Running it, the activity page, settings, fixing problems, Windows
docs/HL7-INTERFACE.md	Exactly what the hospital receives and what it may send (ADT), with samples
docs/ARCHITECTURE.md, DECISIONS.md	The design and the reasons behind it
src/openbedside/	The gateway, about 3,100 lines of Python, standard library only
src/openbedside/examples/acme_json.py	The worked example: a fictional pump and its complete adapter
tests/	36 automated tests
LICENSE, NOTICE, DISCLAIMER.md	Apache 2.0, attribution, and the not-a-medical-device statement
docs/pdf/	Every guide above as a PDF, plus the combined OpenBedside Handbook

Source, releases and issues: github.com/danielpettus/openbedside. Questions and improvements: Daniel C. Pettus, through danielpettus.com.